

MANUAL OPERASIONAL

Sistem Change Data Capture Manajemen Cuti Karyawan

PT Telkom Infrastruktur Indonesia

Versi 1.0 — Agustus 2026

Disusun oleh: Yudha Setiawan Wicaksono

Daftar Isi

Daftar Isi	2
1. Gambaran Umum.....	4
1.1 Tujuan Dokumen	4
1.2 Ruang Lingkup Sistem.....	4
1.3 Audiens	4
2. Arsitektur Sistem	4
2.1 Diagram Alur	4
2.2 Komponen Sistem.....	5
2.3 Alur Data.....	5
3. Persiapan & Instalasi.....	7
3.1 Prasyarat.....	7
3.2 Struktur Direktori.....	7
3.3 Langkah Instalasi	7
3.4 Registrasi Connector	8
4. Konfigurasi	8
4.1 Konfigurasi Debezium Source Connector	8
4.2 Konfigurasi JDBC Sink Connector.....	9
4.3 Konfigurasi Python Merge Service.....	9
4.4 Menambahkan Tabel Baru ke Pipeline	10
5. Operasional Harian	12
5.1 Menjalankan & Menghentikan Sistem.....	12
5.2 Memeriksa Status Komponen	12
5.3 Memeriksa Log	12
6. Monitoring	12
6.1 Log Ringkasan Merge Service	12
6.2 Indikator Sehat vs Bermasalah.....	13
6.3 Latensi sebagai Sinyal Kesehatan	13
7. Troubleshooting	14
8. Keterbatasan yang Perlu Diketahui.....	15
8.1 Greenplum 6 Tidak Mendukung Upsert Native.....	15
8.2 Sistem Berjalan sebagai Micro-batch, Bukan Streaming Murni	15
8.3 Stabilitas Jaringan Berdampak Langsung ke Latensi	15
8.4 Cakupan Saat Ini Terbatas pada Sistem Cuti	15

9. Kontak & Eskalasi	15
Lampiran	17
Lampiran A — Contoh Konfigurasi Lengkap	Error! Bookmark not defined.
Lampiran B — Referensi Perintah Docker yang Sering Dipakai	Error! Bookmark not defined.
Lampiran C — Riwayat Revisi Dokumen	Error! Bookmark not defined.

1. Gambaran Umum

1.1 Tujuan Dokumen

Dokumen ini ditujukan bagi tim Data Management untuk tujuan operasional, pemeliharaan, penelitian, dan mengembangkan lebih lanjut pipeline Change Data Capture (CDC) pada sistem manajemen cuti karyawan. Dokumen ini melengkapi laporan Kerja Praktik yang menjelaskan latar belakang dan proses perancangan sistem; fokus dokumen ini adalah panduan praktis untuk operasional sehari-hari.

1.2 Ruang Lingkup Sistem

Sistem yang dibahas dalam dokumen ini ditujukan untuk *streaming* data (insert, update, delete) dari basis data operasional PostgreSQL sistem manajemen cuti karyawan ke Greenplum sebagai data warehouse, secara near real-time menggunakan pendekatan log-based Change Data Capture. Sistem terdiri atas empat komponen utama: Debezium, Apache Kafka, JDBC Sink Connector, dan Python Job.

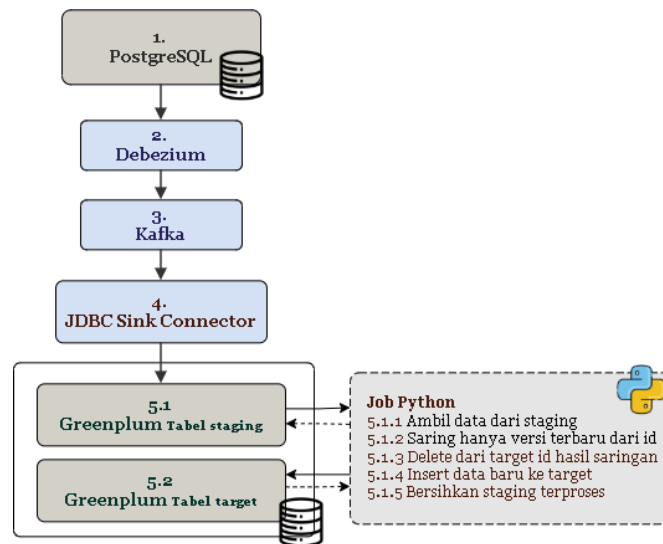
1.3 Audiens

- Administrator/Ops: menjalankan, menghentikan, dan memantau kondisi pipeline harian.
- Data Management: menambahkan tabel atau sistem sumber baru ke pipeline, menyesuaikan konfigurasi. Troubleshooting ketika terjadi anomali (data terlambat, data tidak sampai, dsb).

2. Arsitektur Sistem

2.1 Diagram Alur

Diagram berikut menggambarkan alur data end-to-end, dari basis data sumber hingga tabel target di Greenplum.



Gambar 2.1 Diagram alur pipeline CDC

2.2 Komponen Sistem

Nama Container	Peran	Teknologi
kafka	Message broker — menyalurkan event perubahan sebagai topic.	Apache Kafka 3.6.0
debezium-connect	Source connector — membaca perubahan dari WAL PostgreSQL.	Debezium 2.7 (Kafka Connect)
jdbc-connect	Sink connector — menulis event ke table staging Greenplum.	JDBC Sink Connector 10.7.4 (Confluent)
cdc-service	Merge service — menghapus dan menggabungkan data duplikat, lalu memindahkan data dari staging ke target. Log based monitoring — menjalankan registry dan watchdog untuk mengatasi permasalahan yang disebabkan ketidakstabilan koneksi, dan deteksi perubahan skema.	Python

2.3 Alur Data

1. Perubahan data terjadi pada basis data PostgreSQL sistem cuti.
2. Debezium membaca perubahan tersebut langsung dari Write-Ahead Log (WAL) postgresQL.
3. Setiap perubahan dipublikasikan sebagai event ke topic Kafka yang sesuai.
4. JDBC Sink Connector mengonsumsi event dari Kafka dan menuliskannya secara append-only (insert-only) ke tabel staging di Greenplum.

5. Python Job berjalan berkala (setiap 15 detik) untuk memproses tabel staging: melakukan deduplikasi, penyaringan out-of-order, lalu memindahkan hasil akhirnya ke tabel target melalui pola DELETE-lalu-INSERT.
6. Data pada tabel target di Greenplum merepresentasikan kondisi terkini dari basis data sumber, dengan latensi near real-time.

3. Persiapan & Instalasi

3.1 Prasyarat

- Docker dan Docker Compose sudah terpasang pada host deployment.
- Akses jaringan terbuka dari host ke basis data sumber (PostgreSQL) dan basis data target (Greenplum).
- Kredensial basis data sumber dan target (user, password) dengan hak akses yang memadai — termasuk hak REPLICATION pada PostgreSQL untuk kebutuhan logical decoding.
- Publication pada PostgreSQL sudah dibuat untuk tabel-tabel yang akan direplikasi (lihat bagian 4.1).

3.2 Struktur Direktori

Struktur proyek disusun sebagai berikut:

```
cdc-pipeline-greenplum/
├── docker-compose.yml
├── debezium-connect/
│   ├── Dockerfile
│   └── postgres-connector.json
├── jdbc-connect/
│   ├── Dockerfile
│   └── sink-connector.json
├── cdc-service/ # python job
│   ├── Dockerfile
│   ├── main.py
│   ├── config.py
│   ├── merge.py # merge service
│   └── checks/
│       ├── connector.py # registry & watchdog connector
│       ├── common.py
│       └── schema/
├── helper/ # auto-create scheme staging
│   ├── dashboard/
│   └── testing/
└── Sistem-Cuti/
```

3.3 Langkah Instalasi

1. Salin/clone struktur proyek ke host deployment.
2. Sesuaikan kredensial dan parameter koneksi (host, port, user, password) pada berkas konfigurasi — lihat bagian 4.
3. Jalankan seluruh komponen dengan Docker Compose:

```
docker compose up -d
```

4. Verifikasi seluruh container berjalan dengan status “Up”/“Running”/“Healthy”:

```
docker ps
```

3.4 Registrasi Connector

Debezium source connector dan JDBC Sink Connector pada dasarnya perlu didaftarkan ke Kafka Connect melalui REST API. Namun dalam sistem ini keduanya sudah berjalan otomatis lewat registry pada cdc-service. Jika nantinya registry tidak berjalan otomatis maka curl berikut bisa dijalankan setelah container berjalan:

```
curl -X POST -H "Content-Type: application/json" \
  --data @debezium/postgres-connector.json \
  http://localhost:8085/connectors

curl -X POST -H "Content-Type: application/json" \
  --data @connectors/sink-connector.json \
  http://localhost:8086/connectors
```

Verifikasi status connector setelah registrasi:

```
curl http://localhost:8085/connectors/cutidev-gp-source/statuscurl
curl http://localhost:8086/connectors/greenplum-gp-poc-sink/status
```

4. Konfigurasi

4.1 Konfigurasi Debezium Source Connector

Berkas `debezium/postgres-connector.json` mengatur bagaimana Debezium membaca perubahan dari PostgreSQL:

```
{
  "name": "cutidev-gp-source",
  "config": {
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
    "database.hostname": "<host_postgresql>",
    "database.port": "5432",
    "database.user": "<user>",
    "database.password": "<password>",
    "database.dbname": "<nama_database>",
    "topic.prefix": "<prefix_topic>",
    "plugin.name": "pgoutput",
    "publication.name": "cuti_publication",
    "publication.autocreate.mode": "disabled",
    "slot.name": "debezium_cutidev_gp_poc",
    "schema.include.list": "public",
    "table.include.list": "public.leave_leavedocument,...",
    "snapshot.mode": "initial",
    "tombstones.on.delete": "false",
    "transforms": "unwrap",
    "transforms.unwrap.type":
      "io.debezium.transforms.ExtractNewRecordState",
    "transforms.unwrap.drop.tombstones": "true",
    "transforms.unwrap.delete.handling.mode": "rewrite",
    "transforms.unwrap.add.fields": "op,source.ts_ms"
  }
}
```


Parameter yang perlu diperhatikan saat konfigurasi:

Parameter	Keterangan
table.include.list	Daftar tabel sumber yang direplikasi. Tabel baru harus ditambahkan di sini (lihat 4.4).
publication.name / slot.name	Harus konsisten dengan publication yang dibuat di PostgreSQL; slot bersifat unik per connector.
transforms.unwrap.delete.handling.mode	Diset “rewrite” agar event delete tetap terkirim sebagai baris dengan penanda __deleted = true, bukan tombstone yang dibuang.
transforms.unwrap.add.fields	Menyertakan metadata op (jenis operasi) dan source.ts_ms (timestamp WAL) — keduanya dipakai Python Job untuk dedup dan out-of-order guard.

4.2 Konfigurasi JDBC Sink Connector

connectors/sink-connector.json mengatur penulisan event dari Kafka ke staging Greenplum:

```
{
  "name": "greenplum-gp-poc-sink",
  "config": {
    "connector.class":
      "io.confluent.connect.jdbc.JdbcSinkConnector",
    "tasks.max": "1",
    "topics": "<prefix>.public.leave_leavedocument,...",
    "connection.url":
      "jdbc:postgresql://<host_greenplum>:6432/<db>",
    "connection.user": "<user>",
    "connection.password": "<password>",
    "insert.mode": "insert",
    "auto.create": "true",
    "auto.evolve": "true",
    "table.name.format": "<schema>.stg_${topic}",
    "delete.enabled": "false",
    "transforms": "stripPrefix",
    "transforms.stripPrefix.type":
      "org.apache.kafka.connect.transforms.RegexRouter",
    "transforms.stripPrefix.regex": "<prefix>\\.\\.public\\.\\.\\.\\.*)",
    "transforms.stripPrefix.replacement": "$1"
  }
}
```

Penting, karena keterbatasan Greenplum 6 tidak mendukung klausa "INSERT ... ON CONFLICT" maka insert.mode wajib *insert*, bukan *upsert*. Penggunaan mode *upsert* akan menyebabkan connector gagal dengan error sintaks. Semua logika upsert ditangani terpisah oleh Python Job (bagian 4.3), bukan oleh connector ini.

delete.enabled diset “false” karena event delete ditangani sebagai baris bertanda deleted=true (lihat 4.1), bukan lewat mekanisme delete native connector.

4.3 Konfigurasi Python Merge Service

Seluruh parameter Python merge service diatur melalui environment variable pada service **cdc-service** di docker-compose. Parameter utama:

Parameter	Default	Keterangan
TABLES	leave_leavedocument,leave_historyleave,account_user	Daftar tabel yang diproses, dipisahkan koma
INTERVAL_MERGE	15	Jeda (detik) setelah satu siklus selesai, sebelum siklus berikutnya dimulai
INTERVAL_VACUUM	3600	Jeda (detik) antar operasi VACUUM pada tabel target dan staging
TARGET_SCHEMA	uc_magang	Skema tujuan di Greenplum
ENABLE_DELETED_LOG	false	Bila true, snapshot baris yang dihapus dicatat ke tabel deleted_log
INTERVAL_REGISTRY	300	Jeda (detik) pemeriksaan pendaftaran connector
INTERVAL_WATCHDOG	60	Jeda (detik) pemeriksaan task connector yang gagal
INTERVAL_SCHEMA	300	Jeda (detik) pemeriksaan perubahan skema

4.4 Menambahkan Tabel Baru ke Pipeline

Untuk mereplikasi tabel sumber baru, lakukan empat langkah berikut secara berurutan:

1. Tambahkan nama tabel ke publication PostgreSQL

```
ALTER PUBLICATION cuti_publication ADD TABLE public.nama_tabel_baru
```

Perintah ini hanya dapat dijalankan oleh pemilik publication. Apabila muncul error “must be owner of publication”, penambahan harus dimintakan kepada pemilik publication atau administrator basis data. Tanpa langkah ini, perubahan pada tabel tersebut tidak akan pernah terkirim.

2. Tambahkan tabel ke `table.include.list` pada konfigurasi Debezium (4.1), lalu reload connector.
3. Tambahkan topic yang sesuai ke daftar topics pada JDBC Sink Connector (4.2), lalu reload connector.
4. Buat tabel target di Greenplum secara manual.

```
CREATE TABLE uc_magang.nama_tabel_baru
(LIKE uc_magang.stg_nama_tabel_baru INCLUDING DEFAULTS)
DISTRIBUTED BY (id);
```

Fitur `auto.create` **hanya berlaku untuk tabel staging**. Tabel target tidak pernah dibuat secara otomatis. Apabila tabel target belum ada, merge service akan melewatinya secara diam-diam tanpa menimbulkan galat. Langkah ini dijalankan setelah tabel staging terbentuk, yaitu setelah ada minimal satu baris data yang mengalir.

5. Tambahkan nama tabel ke variabel `TABLES` pada service **cdc-service** di `docker-compose.yml`, lalu jalankan ulang service tersebut:

```
Docker compose up -d cdc-service
```

Reload connector dilakukan lewat REST API Kafka Connect:

```
curl -X POST http://localhost:8085/connectors/cutidev-gp-source/restart
curl -X POST http://localhost:8086/connectors/greenplum-gp-poc-sink/restart
```

5. Operasional Harian

5.1 Menjalankan & Menghentikan Sistem

Perintah	Fungsi
<code>docker compose up -d</code>	Menjalankan seluruh komponen di background.
<code>docker compose down</code>	Menghentikan dan membersihkan seluruh container.
<code>docker compose restart <nama_container></code>	Merestart satu komponen tertentu tanpa mengganggu yang lain.
<code>docker compose logs -f <nama_container></code>	Menampilkan log komponen tertentu secara real-time.

5.2 Memeriksa Status Komponen

Status seluruh container dapat diperiksa dengan:

```
docker ps
```

Status masing-masing connector Kafka Connect (Debezium dan JDBC Sink) diperiksa lewat REST API:

```
curl http://localhost:8085/connectors/cutidev-gp-source/status
curl http://localhost:8086/connectors/greenplum-gp-poc-sink/status
```

Status “RUNNING” pada field connector dan seluruh task menandakan komponen sehat. Status “FAILED” memerlukan pemeriksaan log lebih lanjut (lihat Bab 7).

5.3 Memeriksa Log

—Log Debezium — memuat aktivitas pembacaan WAL dan status replication slot.

```
docker logs cdc-pipeline-greenplum
```

—Log JDBC Sink Connector — memuat aktivitas penulisan ke tabel staging.

```
docker logs jdbc-connect
```

—Log Python Job — memuat ringkasan tiap siklus merge (lihat 6.1).

```
docker logs cdc-service
```

6. Monitoring

6.1 Log Ringkasan Merge Service

Setiap siklus, Python Job mencatat ringkasan dalam format berikut pada log cdc-service:

```
2026-08-13 09:59:07 | [merge] [leave_leavedocument] 3 baris staging diproses, 3
dibersihkan dari staging, 0 baris lama dihapus, 3 baris baru/update ditulis,
latensi min=20.4s avg=20.5s max=20.7s
```

Komponen yang perlu diperhatikan pada tiap baris log: jumlah baris diproses (indikasi ada tidaknya perubahan pada siklus tersebut), jumlah baris yang dibersihkan dari staging (harus konsisten dengan jumlah diproses), dan nilai latensi min/avg/max sebagai indikator kesehatan siklus.

Siklus yang tidak menemukan data pada staging hanya mencatat satu baris penutup:

```
2026-08-13 09:59:22 | [merge] siklus selesai, semua tabel diperiksa
```

6.2 Indikator Sehat dan Bermasalah

Indikator	Kondisi Sehat	Kondisi Bermasalah
Status connector (REST API)	RUNNING pada connector dan seluruh task	FAILED pada connector atau task manapun
Baris staging yang diproses	Bertambah wajar mengikuti volume transaksi	Menumpuk terus tanpa berkurang antar siklus
Latensi rata-rata per siklus	Sekitar 15–30 detik (mendekati interval siklus)	Melonjak hingga puluhan/ratusan detik secara konsisten
Log Python Job	Muncul rutin setiap interval <code>merge_interval</code>	Berhenti muncul (proses macet) atau memuat <code>traceback error</code>

6.3 Latensi sebagai Sinyal Kesehatan

Berdasarkan pengujian pada 185 siklus, latensi rata-rata pada kondisi jaringan normal berada di kisaran 19,0 detik (median 14,6 detik), dengan 86% siklus bervariasi di bawah 60 detik. Angka ini dapat dijadikan baseline operasional, bila latensi rata-rata secara konsisten berada jauh di atas kisaran tersebut, kemungkinan besar penyebabnya adalah gangguan konektivitas jaringan menuju basis data sumber atau target, bukan lonjakan volume data (uji korelasi Pearson pada pengujian sebelumnya menunjukkan $r^2 = 0,046$, yang berarti volume data hanya menjelaskan sebagian kecil variasi latensi). Prioritaskan pemeriksaan konektivitas jaringan sebelum menambah resource komputasi.

7. Troubleshooting

Tabel berikut merangkum gejala umum yang dapat ditemui saat mengoperasikan pipeline, beserta kemungkinan penyebab dan langkah penanganannya.

Gejala	Kemungkinan Penyebab	Langkah Penanganan
Data tidak sampai ke tabel target	Connector Debezium/JDBC Sink berhenti (status FAILED), atau Python Job berhenti berjalan.	Cek status connector via REST API (5.2). Cek log cdc-service apakah proses masih berjalan. Restart komponen yang FAILED.
Latensi melonjak drastis di banyak siklus berturut-turut	Gangguan konektivitas jaringan menuju basis data sumber atau target (bukan volume data — lihat 6.3).	Periksa konektivitas jaringan (ping/telnet ke host source dan target) sebelum menambah resource. Cek juga status VPN/firewall bila relevan.
Kolom baru pada tabel sumber tidak muncul di tabel staging	auto.evolve belum aktif pada JDBC Sink Connector, atau connector belum memuat ulang metadata skema.	Pastikan auto.evolve=true pada konfigurasi (4.2). Restart JDBC Sink Connector agar metadata skema dimuat ulang.
Data terlihat duplikat di tabel target	Siklus merge terhenti di tengah proses (mis. container mati saat DELETE-INSERT berjalan), atau guard out-of-order gagal.	Cek log cdc-service untuk siklus yang gagal/tidak selesai. Jalankan ulang siklus merge secara manual bila diperlukan; pola DELETE-INSERT bersifat idempoten sehingga aman dijalankan ulang.
Tabel baru yang didaftarkan tidak kunjung terisi	Salah satu dari empat langkah pada 4.4 terlewat (publication, table.include.list, topics, atau merge-config.py).	Telusuri ulang keempat langkah pada 4.4 secara berurutan; kesalahan paling umum adalah lupa menambahkan tabel ke publication PostgreSQL.

8. Keterbatasan yang Perlu Diketahui

Beberapa karakteristik arsitektur berikut bersifat inheren terhadap desain sistem saat ini, dan penting dipahami sebelum melakukan perubahan atau pengembangan lebih lanjut.

8.1 Greenplum 6 Tidak Mendukung Upsert

Greenplum versi 6 dibangun di atas kernel PostgreSQL 9.4, yang mendahului fitur INSERT ... ON CONFLICT (baru tersedia di PostgreSQL 9.5) maupun perintah MERGE. Karena itu, seluruh logika upsert pada pipeline ini diimplementasikan secara manual melalui Python Job dengan pola DELETE-lalu-INSERT, bukan lewat fitur native basis data. Perubahan pada `insert.mode` JDBC Sink Connector menjadi “upsert” akan menyebabkan kegagalan connector.

8.2 Sistem Berjalan sebagai Micro-batch, Bukan Streaming Murni

Sistem CDC hanya bisa melakukan *insert* karena keterbatasan basis data Greenplum. Python Job memproses tabel staging secara berkala (default setiap 15 detik), bukan per-event secara instan. Ini berarti terdapat latensi minimum yang inheren terhadap desain, terlepas dari kondisi jaringan. Memperpendek interval siklus dapat menurunkan latensi, namun juga meningkatkan beban query terhadap Greenplum — perlu pertimbangan trade-off sebelum diubah.

8.3 Stabilitas Jaringan Berdampak Langsung ke Latensi

Sebagaimana dijelaskan pada 6.3, latensi pipeline lebih dipengaruhi oleh stabilitas jaringan dibandingkan volume data. Pada deployment production dengan koneksi jaringan yang kurang stabil, latensi dapat meningkat signifikan meski volume transaksi rendah.

8.4 Cakupan Saat Ini Terbatas pada Sistem Cuti

Pipeline ini baru mencakup sistem manajemen cuti karyawan sebagai tahap awal implementasi. Ekspansi ke sistem operasional lain memerlukan evaluasi ulang terhadap karakteristik data dan volume masing-masing sistem, mengikuti pola pada Bab 4.4.

9. Kontak & Eskalasi

Tabel berikut memuat pihak yang dapat dihubungi apabila ditemukan gangguan yang tidak dapat diselesaikan lewat langkah-langkah pada Bab 7.

Peran	Nama	Kontak
Developer	Yudha Setiawan Wicaksono	yudha.sw2006@gmail.com

Peran	Nama	Kontak

Lampiran